# Learning Objectives

**Learners will be able to…**

- **Describe the structure of hash tables**
- **Identify the importance of hashing**
- **Implement a basic hash table including collision resolution**
- **Create hashing functions that reduce collisions**

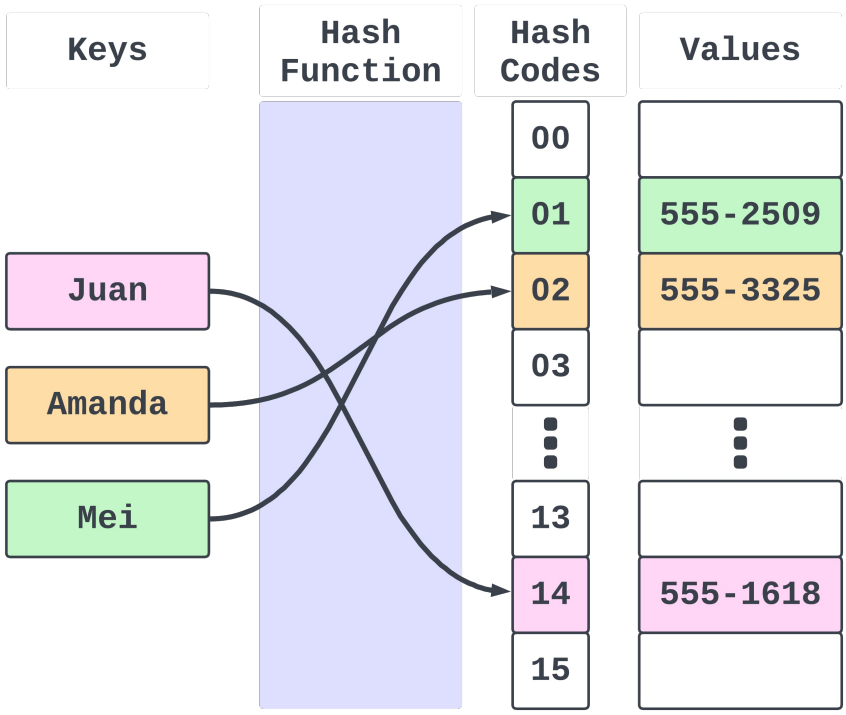
# Hash Tables

## What are Hash Tables?

A **hash table**, also known as a hash map, is an efficient data structure that pairs keys with values and offers quick data retrieval. This may sound similar to an array, which has an index that is associated with an element. However, there is an important difference between a hash table and an array; keys are hashed.

**Hashing** is a pivotal concept in computer science, central to the functionality of hash tables. It involves transforming data of arbitrary size, such as a text string, password, or file, into a fixed-length value known as a hash code – akin to a unique fingerprint for a piece of information. This process operates through three main stages:

1. **Input:** The process begins with the raw data.
2. **Hash Function:** A specialized function processes this data, employing an algorithm to scramble it into a fixed-length hash code. This function can be likened to a magical blender, ensuring that regardless of the input size, the output – the hash code – remains consistent in size.
3. **Hash Table:** This data structure, reminiscent of a filing cabinet or buckets with folders labeled by hash codes, stores key-value pairs. The key is the hash code, and the value is the original data.



The image above represents a hash table of individuals (key) and their phone numbers (value). The key is run through a hashing function, which converts the name into a hash code. The value is then stored in a slot in the hash table determined by the hash code.

The effectiveness of hash tables hinges on the design of the hash function, which should swiftly and reliably produce a hash code. This code determines the storage or search location for data in the structure. Hash functions are deterministic, which means a given input will always produce the same output. That means if you need to look up the phone number for Juan, hash the name, and then look at the data stored at that hash code. Hashing allows for constant-time data access, a stark advantage for operations like search, insert, or delete.

## Benefits of Hashing

Hashing provides several advantages over storing the original data. Some of the most common benefits are:

- **Fast Data Retrieval:** Leveraging a hash code to search in a hash table drastically reduces the time compared to sifting through the entire original dataset.
- **Efficient Storage:** Hash codes, typically smaller than the original data, help conserve storage, particularly for sizable datasets.
- **Data Integrity:** Certain hash functions can detect alterations in the original data since any modification results in a new, distinct hash code. This feature is invaluable for security purposes.

## Applications of Hashing

Hashing is such a powerful tool that it is used in other places besides just hash tables:

- **Databases:** Quick data indexing and retrieval, like finding a user by their username.
- **Password Storage:** Storing passwords as hash codes for enhanced security.
- **Digital Signatures:** Ensuring the authenticity and integrity of documents.
- **Error Detection:** Verifying data integrity during transmission or storage by comparing hash codes.

However, hashing is not a perfect process. It is crucial to be aware of these potential issues:

- **Collisions:** The rare possibility of different data yielding identical hash codes. Effective hash functions aim to minimize this risk, which can degrade performance.
- **Irreversibility:** Typically, reconstructing original data from a hash code is infeasible.

## Hash Functions and Operations

Hash functions serve as the backbone, converting information into a compact hash. They're deterministic, ensuring consistent hashes for the same data, and ideally unique, to prevent data overlap. Hashing offers increased efficiency, so these operations have a near constant time complexity of $O(1)$. Key operations in hash tables include:

1. **Insertion (Put):** Places a new key-value pair in the table, handling collisions through methods like chaining or open addressing.
2. **Deletion (Remove):** Removes a pair based on its key.
3. **Search (Get):** Retrieves a value using the key to calculate its index.

In essence, hash tables, underpinned by the principle of hashing, offer a robust framework for efficient data management and access, marking a cornerstone of effective computing. However, factors such as a suboptimal hash function or a high load factor can degrade this efficiency. Unlike data structures like stacks, hash tables don't inherently maintain element order, though specific variations can retain insertion order.

# Implementing a Basic Hash Table

## Coding a Hash Table

The hash table data structure can be implemented using various underlying methods, with arrays being one of the most common. In C++, a simple hash table can be implemented using an array for storage and a hashing function to translate keys into valid array `indices`. We will also cover the common `put` (insert), `get` (search), and `remove` (delete) operations of a hash table.

### Hash Table

The first step is to create the general structure of the hash table. The `HashTable` class has the `maxSize` attribute that dictates the number of entries that can be stored in the hash table. You have two arrays, one for keys and the other for values. Notice how both arrays are of type `int*` (pointer to int). Using pointers allows us to use `nullptr` which will help simplify our code for the operations.

Create a constructor that takes an integer. Set `maxSize` to the given integer. Then create arrays of size `maxSize` for both `keys` and `values`. Remember, C++ will use the default values for each array. That means both arrays are filled with `nullptr`.

```
class HashTable {
private:
    int* keys;
    int* values;
    int maxSize;

public:
    HashTable(int size) {
        maxSize = size;
        keys = new int[maxSize];
        values = new int[maxSize];
        for (int i = 0; i < maxSize; ++i) {
            keys[i] = -1; // Use -1 to indicate empty slot
            values[i] = 0;
        }
    }
};
```

### Hash Function

For this example, we are going to create a simple hash function. Remember, we are using an array to represent our hash table, so we need to make sure that our hash function returns a valid index. To do this, we are going to use the modulo operator. This ensures that every value returned (`0` to `maxSize - 1`) from the hash function is a valid index.

```
private:
    int hashFunction(int key) {
        return key % maxSize;
    }
```

## Insertion

The `put` method inserts the key-value pairs into the hash table. As such, it takes integers representing the key and value. Using the `hashFunction` method, transform the key into a hash code. We are going to call this `hashIndex`, and it will be used to find elements in both the key and value arrays.

```
public:
    void put(int key, int value) {
        int hashIndex = hashFunction(key);
        keys[hashIndex] = key;
        values[hashIndex] = value;
    }
```

## Search

Unlike an array, you do not need to iterate over the data structure to find a key-value pair. Instead, call the hash function on the given key and use this index to return the associated element from the values array. This exemplifies the performance benefits of a hash table.

```
public:
    int get(int key) {
        int hashIndex = hashFunction(key);
        return values[hashIndex];
    }
```

## Deletion

Deletion is done by setting elements in the `keys` and `values` arrays back to their default values. Run the given key through the hash function to generate an index. Use this index to set the element in the `keys` array back to `-1` and the element in the `values` array back to `0`. This slot in the hash table is now seen as “empty”.

```
public:
    void remove(int key) {
        int hashIndex = hashFunction(key);
        keys[hashIndex] = -1;
        values[hashIndex] = 0;
    }
```

## Testing Our Code

Add the main method to your program which creates an instance of the `HashTable` class. In addition, uses the `put`, `get`, and `remove` methods:

```
int main() {
    HashTable myHashTable(10);

    myHashTable.put(1, 100);
    myHashTable.put(2, 200);
    myHashTable.put(3, 300);

    std::cout << "Get value for key 2: " << myHashTable.get(2)
        << std::endl;
    myHashTable.remove(2);
    std::cout << "Get value for key 2 after removal: " <<
        myHashTable.get(2) << std::endl;

    return 0;
}
```

▼ **Code**

Your code should look like this:

```
class HashTable {
private:
    int* keys;
    int* values;
    int maxSize;

public:
    HashTable(int size) {
        maxSize = size;
        keys = new int[maxSize];
        values = new int[maxSize];
        for (int i = 0; i < maxSize; ++i) {
            keys[i] = -1; // Use -1 to indicate empty slot
            values[i] = 0;
        }
    }

private:
    int hashFunction(int key) {
        return key % maxSize;
    }
```

```
public:
    void put(int key, int value) {
        int hashIndex = hashFunction(key);
        keys[hashIndex] = key;
        values[hashIndex] = value;
    }

    int get(int key) {
        int hashIndex = hashFunction(key);
        return values[hashIndex];
    }

    void remove(int key) {
        int hashIndex = hashFunction(key);
        keys[hashIndex] = -1;
        values[hashIndex] = 0;
    }
};

int main() {
    HashTable myHashTable(10);

    myHashTable.put(1, 100);
    myHashTable.put(2, 200);
    myHashTable.put(3, 300);

    std::cout << "Value for key 2: " << myHashTable.get(2) <<
        std::endl;
    myHashTable.remove(2);
    std::cout << "Value for key 2 after removal: " <<
        myHashTable.get(2) << std::endl;

    return 0;
}
```

You should see the following output:

```
Value for key 2: 200
Value for key 2 after removal: 0
```

This is a basic example of a hash table built with arrays that performs common operations. The nature of hash tables means that these operations are, on average, very fast:

- Insert (`put`): $O(1)$ on average
- Delete (`remove`): $O(1)$ on average
- Search (`get`): $O(1)$ on average

However, this hash table is simple to a fault. Let's update the `main` method to the code below. In this instance, the program inserts the keys `12`, `2`, and `3`. It then gets the value associated with key `12`, deletes key `2`, and prints the

value at key `12` one more time.

```
int main() {
    HashTable myHashTable(10);

    myHashTable.put(12, 1200);
    myHashTable.put(2, 200);
    myHashTable.put(3, 300);

    std::cout << "Value for key 12: " << myHashTable.get(12) <<
        std::endl;
    myHashTable.remove(2);
    std::cout << "Value for key 12 after removal: " <<
        myHashTable.get(12) << std::endl;

  return 0;
}
```

This hash table does not handle collisions. Right now, all it does is perform an operation using index generated by the hash function. Both `12 % 10` and `2 % 10` produce the same result – 2. Because of this, the key-value pair `12, 1200` is overwritten by the key-value pair `2, 200`. This explains why the first print statement prints `200` instead of `1200`. Once you remove key `2`, there is nothing in this slot in the hash table, which explains why the second print statement produces 0.

# Handling Collisions - Linear Probing

## Open Addressing with Linear Probing

To improve the performance of our hash table, we need to be able to handle collisions from the hash function. There are several ways of doing this, but we are going to focus on open addressing and linear probing. Open addressing is a collision resolution strategy that looks for an open slot in the hash table. Linear probing is the process of advancing linearly through the hash table looking for an open slot.

While the two terms are often used interchangeably, it is better to think of linear probing as a specific type of open addressing.

We are going to modify the `put`, `get`, and `remove` methods to handle collisions. The constructor and `hashFunction` method do not need to be altered.

### Updating Insertion

For the `put` method, we need to insert a loop after calling the hash function and before updating the values in the `keys` and `values` arrays. This loop runs only if there is a collision. We have a collision when the element at `keys[hashIndex]` is not -1 and it is not the value of the given key. When there is a collision, increment `hashIndex` by 1. However, this could lead to an index out of range error, so use modulo to ensure that the index is valid.

```
    void put(int key, int value) {
        int hashIndex = hashFunction(key);
        int numberOfProbes = 0;

        while (keys[hashIndex] != -1 && keys[hashIndex] != key)
        {
            hashIndex = (hashIndex + 1) % maxSize;
        }

        keys[hashIndex] = key;
        values[hashIndex] = value;
    }
```

However, there is still a problem with this code. Imagine if the hash table is entirely full. You then try to insert an additional element. The program would encounter a timeout error because the linear probing would never end. We need to avoid infinite loops. Create the variable `numberOfProbes` and set it to `0`. Every time you increment the index in linear probing,

increment `numberOfProbes`. If the number of probes is greater than the maximum size of the hash table, then you have checked every slot. Exit the method with a `return` statement.

```
    void put(int key, int value) {
        int hashIndex = hashFunction(key);
        int numberOfProbes = 0;

        while (keys[hashIndex] != -1 && keys[hashIndex] != key)
        {
            hashIndex = (hashIndex + 1) % maxSize;
            numberOfProbes++;

            if (numberOfProbes > maxSize) {
                return;
            }
        }

        keys[hashIndex] = key;
        values[hashIndex] = value;
    }
```

## Updating Search

The updates to the `get` method are going to be very similar to the updates to the `put` method. We need to keep track of the number of probes in order to avoid an infinite loop when the hash table is full. Just as before, use a while loop to probe the hash table for an empty slot in case of a collision. Return `-1` if you have probed the entire hash table. If the loop ends, check to see if the current element in the `keys` array is `-1`. That means the key is not in the hash table, return `-1`. Otherwise, return the element at `values[hashIndex]`.

```
    int get(int key) {
        int hashIndex = hashFunction(key);
        int numberOfProbes = 0;

        while (keys[hashIndex] != -1 && keys[hashIndex] != key)
        {
            hashIndex = (hashIndex + 1) % maxSize;
            numberOfProbes++;

            if (numberOfProbes > maxSize) {
                return -1;
            }
        }

        if (keys[hashIndex] == -1) {
            return -1;
        } else {
            return values[hashIndex];
        }
    }
```

### Updating Deletion

Updates to the `remove` method are going to be a bit different compared to the other two. You will still keep track of the number of probes. However, the while loop will run as long as the value of `keys[hashIndex]` is not -1. If you find the matching key, set the elements in the `keys` and `values` arrays back to their default values. If the current element is not the target key, continue probing. Exit the method if the number of probes exceeds the maximum size.

```
    void remove(int key) {
        int hashIndex = hashFunction(key);
        int numberOfProbes = 0;

        while (keys[hashIndex] != -1) {
            if (keys[hashIndex] == key) {
                keys[hashIndex] = -1;
                values[hashIndex] = 0;
                return;
            }

            hashIndex = (hashIndex + 1) % maxSize;
            numberOfProbes++;

            if (numberOfProbes > maxSize) {
                return;
            }
        }
    }
```

## Testing Our Code

Update the `main` method to test our edge cases. We want to make sure that we avoid an infinite loop when trying to insert a key-value pair into a full hash table, handle collisions, avoid an infinite loop when trying to remove a non-existent key-value pair from a full hash table, and avoid an infinite loop when removing a non-existent key-value pair from a full hash table.

```
int main() {
    HashTable myHashTable(10);

    myHashTable.put(12, 1200);
    myHashTable.put(2, 200);
    myHashTable.put(3, 300);
    myHashTable.put(13, 1300);
    myHashTable.put(7, 7);
    myHashTable.put(37, 37);
    myHashTable.put(122, 122);
    myHashTable.put(123, 123);
    myHashTable.put(124, 124);
    myHashTable.put(125, 125);
    myHashTable.put(1, 1); // edge case for put method

    std::cout << "Value for key 12: " << myHashTable.get(12) <<
        std::endl; // handle collision
    myHashTable.remove(45); // edge case for remove method
    std::cout << "Value for key 57 after removal: " <<
        myHashTable.get(57) << std::endl; // edge case for get
        method

    return 0;
}
```

▼ **Code**

Your code should look like this:

```
class HashTable {
private:
    int* keys;
    int* values;
    int maxSize;

public:
    HashTable(int size) {
        maxSize = size;
        keys = new int[maxSize];
        values = new int[maxSize];
        for (int i = 0; i < maxSize; ++i) {
            keys[i] = -1; // Use -1 to indicate empty slot
            values[i] = 0;
        }
    }

private:
    int hashFunction(int key) {
        return key % maxSize;
    }

public:
    void put(int key, int value) {
        int hashIndex = hashFunction(key);
        int numberOfProbes = 0;
```

```
        while (keys[hashIndex] != -1 && keys[hashIndex] != key)
        {
            hashIndex = (hashIndex + 1) % maxSize;
            numberOfProbes++;

            if (numberOfProbes > maxSize) {
                return;
            }
        }

        keys[hashIndex] = key;
        values[hashIndex] = value;
    }

    int get(int key) {
        int hashIndex = hashFunction(key);
        int numberOfProbes = 0;

        while (keys[hashIndex] != -1 && keys[hashIndex] != key)
        {
            hashIndex = (hashIndex + 1) % maxSize;
            numberOfProbes++;

            if (numberOfProbes > maxSize) {
                return -1;
            }
        }

        if (keys[hashIndex] == -1) {
            return -1;
        } else {
            return values[hashIndex];
        }
    }

    void remove(int key) {
        int hashIndex = hashFunction(key);
        int numberOfProbes = 0;

        while (keys[hashIndex] != -1) {
            if (keys[hashIndex] == key) {
                keys[hashIndex] = -1;
                values[hashIndex] = 0;
                return;
            }

            hashIndex = (hashIndex + 1) % maxSize;
            numberOfProbes++;

            if (numberOfProbes > maxSize) {
                return;
            }
        }
    }

};
int main() {
    HashTable myHashTable(10);
```

```
    myHashTable.put(12, 1200);
    myHashTable.put(2, 200);
    myHashTable.put(3, 300);
    myHashTable.put(13, 1300);
    myHashTable.put(7, 7);
    myHashTable.put(37, 37);
    myHashTable.put(122, 122);
    myHashTable.put(123, 123);
    myHashTable.put(124, 124);
    myHashTable.put(125, 125);
    myHashTable.put(1, 1); // edge case for put method

    std::cout << "Value for key 12: " << myHashTable.get(12) <<
        std::endl; // handle collision
    myHashTable.remove(45); // edge case for remove method
    std::cout << "Value for key 57 after removal: " <<
        myHashTable.get(57) << std::endl; // edge case for get
        method

    return 0;
}
```

You should see the following output:

```
Value for key 12: 1200
Value for key 57 after removal: -1
```

Our hash table now handles collisions with linear probing. This collision resolution strategy is sometimes preferred because it is space-efficient. That is, we do not have to allocate any more memory (aside from the two arrays) to keys with the same hash code. The drawback, however, is that you can encounter edge cases where you can end up in an infinite loop.

# Handling Collisions - Chaining

## Resolving Collisions with a Linked List

Chaining is another collision resolution strategy. Unlike linear probing, chaining does not search for an empty slot. Instead, every slot in the hash table is a linked list (or another dynamic data structure). So when there is a collision, you can just add the key-value pair to the list. Chaining offers simplicity over linear probing, but, as we will see, this comes with a cost.

### Hash Table

Create the `HashTableChaining` class. We are going to create the internal class `Entry` that holds the key-value pair (both of which are integers). Just as before, the hash table has a maximum size. Finally, the table itself will be an array of `linked` lists.

```
class HashTableChaining {
    class Entry {
    public:
        int key;
        int value;

        Entry(int key, int value) : key(key), value(value) {}
    };

    int maxSize;
    std::list<Entry>* table; // Array of linked lists

public:
    HashTableChaining(int size) : maxSize(size) {
        table = new std::list<Entry>[maxSize];
    }

    int hashFunction(int key) {
        return key % maxSize;
    }
};
```

The constructor for the hash table takes an integer representing the maximum size. Create the array of the given size. Then iterate over the array instantiating a new `std::list` object for each element. Similar to our previous examples, the hashing function uses the modulo operator to generate a hash code.

### Insertion

Generate a hash code using the given key. Since this slot uses a linked list, we cannot just add the key-value pair. What if it already exists in the list? In this case, find the node in the list and update the value. If the key-value pair is not in the list, then add a new node to the linked list.

Use `table[hashIndex]` to go to the appropriate slot. Then use a for loop to iterate over the linked list. If you find a matching key, update the value and exit the method. If the loop terminates, add a new `Entry` object (the key-value pair) to the list.

```
    void put(int key, int value) {
        int hashIndex = hashFunction(key);

        for (auto& entry : table[hashIndex]) {
            if (entry.key == key) {
                entry.value = value;
                return;
            }
        }

        table[hashIndex].emplace_back(key, value);
    }
```

## Search

Searching for a key-value pair works in a similar manner. Generate a hash code with the given key. Go to that slot in the hash table. Then iterate over the linked list. If you find a matching key, return the associated value. Otherwise, return `-1` to indicate the specified key does not exist in the hash table.

```
    int get(int key) {
        int hashIndex = hashFunction(key);
        for (const auto& entry : table[hashIndex]) {
            if (entry.key == key) {
                return entry.value;
            }
        }
        return -1;
    }
```

## Deletion

Deleting a key-value pair works a bit differently. Because we are using a linked list, we cannot tell C++ to delete a node at a given index. Instead, we are going to use an iterator to store the node we want to remove. Start just as before by generating a hash code and iterating over the linked list. When we find the matching key, set the iterator to the current node. After the loop ends, check to see if the iterator is not at the end. If so, remove the specified node.

```
    void remove(int key) {
        int hashIndex = hashFunction(key);
        auto& list = table[hashIndex];
        for (auto it = list.begin(); it != list.end(); ++it) {
            if (it->key == key) {
                list.erase(it);
                return;
            }
        }
    }
```

## Testing Our Code

Add a main function to your program. Create a hash table and add three key-value pairs. Test the get and remove methods.

```
int main() {
    HashTableChaining myHashTable(10);

    myHashTable.put(1, 100);
    myHashTable.put(2, 200);
    myHashTable.put(3, 300);

    std::cout << "Get value for key 2: " << myHashTable.get(2)
        << std::endl;
    myHashTable.remove(2);
    std::cout << "Get value for key 2 after removal: " <<
        myHashTable.get(2) << std::endl;

    return 0;
}
```

▼ **Code**

Your code should look like this:

```
#include <iostream>
#include <list>

class HashTableChaining {

    class Entry {
    public:
        int key;
        int value;

        Entry(int key, int value) : key(key), value(value) {}
    };

    int maxSize;
    std::list<Entry>* table; // Array of linked lists

public:
```

```
    HashTableChaining(int size) : maxSize(size) {
        table = new std::list<Entry>[maxSize];
    }

    int hashFunction(int key) {
        return key % maxSize;
    }

    void put(int key, int value) {
        int hashIndex = hashFunction(key);

        for (auto& entry : table[hashIndex]) {
            if (entry.key == key) {
                entry.value = value;
                return;
            }
        }

        table[hashIndex].emplace_back(key, value);
    }

    int get(int key) {
        int hashIndex = hashFunction(key);
        for (const auto& entry : table[hashIndex]) {
            if (entry.key == key) {
                return entry.value;
            }
        }
        return -1;
    }

    void remove(int key) {
        int hashIndex = hashFunction(key);
        auto& list = table[hashIndex];
        for (auto it = list.begin(); it != list.end(); ++it) {
            if (it->key == key) {
                list.erase(it);
                return;
            }
        }
    }
};

int main() {
    HashTableChaining myHashTable(10);
    myHashTable.put(1, 100);
    myHashTable.put(2, 200);
    myHashTable.put(3, 300);
    std::cout << "Get value for key 2: " << myHashTable.get(2)
        << std::endl;
    myHashTable.remove(2);
    std::cout << "Get value for key 2 after removal: " <<
        myHashTable.get(2) << std::endl;
    return 0;
}
```

You should see the following output:

```
Get value for key 2: 200
Get value for key 2 after removal: -1
```

▼ **Potential memory issues**

> The code uses a raw pointer `std::list<Entry>*` to manage a dynamic array of linked lists. Although this may not generate specific compiler warnings, it introduces risks associated with manual memory management, such as potential memory leaks and dangling pointers.
>
> Instead of using a raw pointer, use `std::vector<std::list<Entry>` to manage dynamic arrays. `std::vector` provides automatic memory management and bounds checking, which enhances safety and reduces the risk of memory-related issues.

Chaining allows for the dynamic storing of key-value pairs in the event of a collision. There is a simplicity to this method of collision resolution when compared to linear probing. However, hash tables using chaining can use more memory when compared to linear probing. In addition, chaining uses linear search ($O(n)$) to find keys in the linked list for each operation covered.

challenge

**Try this variation**

Update the main method so that the hash table has a maximum size of 5. Then add eight different key-value pairs that all have the same hash code (7).

```
int main() {
    HashTableChaining myHashTable(5);

    myHashTable.put(7, 7);
    myHashTable.put(17, 17);
    myHashTable.put(27, 27);
    myHashTable.put(37, 37);
    myHashTable.put(47, 47);
    myHashTable.put(57, 57);
    myHashTable.put(67, 67);
    myHashTable.put(77, 77);

    std::cout << "Get value for key 7: " <<
        myHashTable.get(7) << std::endl;
    std::cout << "Get value for key 17: " <<
        myHashTable.get(17) << std::endl;
    std::cout << "Get value for key 27: " <<
        myHashTable.get(27) << std::endl;
    std::cout << "Get value for key 37: " <<
        myHashTable.get(37) << std::endl;
    std::cout << "Get value for key 47: " <<
        myHashTable.get(47) << std::endl;
    std::cout << "Get value for key 57: " <<
        myHashTable.get(57) << std::endl;
    std::cout << "Get value for key 67: " <<
        myHashTable.get(67) << std::endl;
    std::cout << "Get value for key 77: " <<
        myHashTable.get(77) << std::endl;

    return 0;
}
```

▼ **What is happening?**

> Because we are using a dynamic data structure for each slot in the hash table, that means the maximum size is related to the number of unique hash codes that can be stored. However, key-value pairs that share a hash code do not count towards the maximum size of the hash table.

Understanding and implementing collision resolution strategies are pivotal to the efficient functioning of a hash table. Chaining is straightforward but requires additional memory for the linked lists. Open addressing with linear probing is more space-efficient but requires careful handling of

insertion and deletion to manage the probe sequence correctly. Experiment with these methods to see how they affect the performance and behavior of your hash table in different scenarios.

# Hash Function

## Improving Performance with Hashing

Hash functions are the heart of hash tables, converting keys into array indices. A good hash function distributes keys uniformly across the hash table to minimize collisions and maximize the efficiency of the hash table operations. In this section, we'll explore the nature of hash functions, their properties, and how to design a good hash function.

### Properties of a Good Hash Function

A good hash function will do the following things:

1. **Deterministic**: The same input should always produce the same output.
2. **Fast Computation**: The hash function should be capable of returning the hash code in a short time.
3. **Uniform Distribution**: The hash codes should be uniformly distributed across the hash table to minimize collisions.
4. **Minimal Collision**: Two different inputs should not produce the same output hash code, although this is not always possible to avoid entirely.

### Designing a Simple Hash Function

A basic hash function for integers could simply return the remainder when the key is divided by the size of the table (modulo operation). This is the hash function that we have been using so far.

```
private:
  static int hashFunction(int key) {
      return key % tableSize;
  }
```

This hash function is easy to understand and produces a valid index. It is also deterministic and fast. However, it does not provide good distribution or minimal collisions. To see the weaknesses of this hash function, copy the following code into the code editor. This code assumes a hash table size of 50 and prints the hash codes for ten numbers selected between 0 and 999.

```
int tableSize = 50;

int hashFunction(int key) {
    return key % tableSize;
}

void printHashValues() {
    int numbers[] = {743, 323, 78, 631, 741, 28, 363, 991, 693,
        304};
    for (int num : numbers) {
        std::cout << "Hash value for " << num << " is: " <<
        hashFunction(num) << std::endl;
    }
}

int main() {
    printHashValues();
    return 0;
}
```

Notice how there are a couple of collisions. Keys 743 and 693 have the same hash code (43) while 741 and 991 have the same hash code (41). This code may be fast and deterministic, but under certain circumstances, it does not provide a wide array of output. This leads to too many collisions. We can take steps to improve upon this hash function.

## Use Prime Numbers

Something simple you can do is to set the size of the table to a prime number. Since prime numbers are only divisible by 1 and themselves, they tend to reduce the patterns in output. Update the `tableSize` variable to be `53` and run the program again. The output from hashing the same keys no longer has a repeating pattern.

```
int tableSize = 53;
```

## Use Constants

Another strategy to improve hashing is to modify the value of the key with constants. That is, you must modify the key the same way every time you hash. Small changes to the key can lead to a better distribution. In the example below, we multiply the key by `1.381` and round up before the modulo operation. Be sure to convert the modified key back to an integer as modulo can only work with integers.

```
int hashFunction(int key) {
    return static_cast<int>(round(1.381 * key)) % tableSize;
}
```

## Mixing Up Bits

If you are working with an integer key, you can mix up the bits in the key before performing the modulo operation. This helps with a more even distribution of hash codes. The given key becomes `h`. Bitwise operations are used to shift bits around and the XOR operator `(^)` functions as a crude randomizer mixing up the bits. So the example below sifts the bits `20` places to the right and `12` places to the right. The XOR operator mixes up the result. This is done again by shifting bits `7` and `4` places to the right. The result is then multiplied by a very large prime number (0x9E3779B9 is the hexadecimal representation of 2,543,191,049). Finally, you perform the modulo operation on the absolute value of the mixed-up number.

```
int hashFunction(int key) {
    int h = key;
    h ^= (h >> 20) ^ (h >> 12);
    h ^= (h >> 7) ^ (h >> 4);
    h *= 0x9E3779B9;
    return std::abs(h) % tableSize;
}
```

## Handling Strings and Complex Data

So far, all of our examples use integers as keys. It is quite common to use strings (or even other data types) as the key in a hash map. When the keys are not integers, the hash function must first convert the key into an integer. For strings, a common approach is to interpret the sequence of characters in the string as a number. In the code below, `hashVal` stores the numerical representation of a string. Iterate over the string, summing the numerical values of each character in the string. We multiply by `37` since prime numbers help with uniform distribution. Then perform modulo on the numeric value. If this value happens to be negative, increment by the size of the table.

```
int hashFunction(const std::string& key) {
    int hashVal = 0;
    for (char ch : key) {
        hashVal = 37 * hashVal + ch;
    }

    hashVal %= tableSize;

    if (hashVal < 0) {
        hashVal += tableSize;
    }
    return hashVal;
}
```

Since we are now hashing strings, we need to update the `printHashValues` method to use strings in the array instead of integers.

```
void printHashValues() {
    std::string strings[] = {"one", "two", "three", "four",
        "five", "six", "seven", "eight", "nine", "ten"};
    for (const auto &s : strings) {
        std::cout << "Hash value for '" << s << "' is: " <<
        hashFunction(s) << std::endl;
    }
}
```

Another method for converting a string key into a hash code is to use a pre-made hash function. C++'s standard library includes the `std::hash` function. This function focuses on speed and distribution, but it is not cryptographically secure.

▼ **Cryptographic hashes**

> Cryptographic hashes are special hashes that have traits that make them ideal for authentication codes, digital signatures, fingerprinting, and other tasks in the information security realm. While cryptographic hashes are robust, they are a bit more complicated than some of the hashing techniques discussed.

`hasher` is a hash function object that returns a hash value for a string. Create the `hashValue` variable which contains the hashed value of the given string. Then cast the hash value into a regular integer. Use `std::abs` to ensure the hash value is positive. Finally, return the remainder when hash value is divided by the size of the table.

```
int hashFunction(const std::string& key) {
    std::hash<std::string> hasher;
    size_t hashValue = hasher(key);

    return std::abs(static_cast<int>(hashValue)) % tableSize;
}
```

A well-designed hash function is crucial for the performance of a hash table. It should be fast, distribute keys uniformly, and minimize collisions. While designing a hash function, consider the types of keys you expect and test the function to ensure it provides a good distribution for your specific data set. Combining a robust hash function with an effective collision resolution strategy can significantly enhance the performance of a hash table.

# Recap

## Reviewing Hash Tables and Hashing

In this module, we delved into the concept of hashing, the structure and implementation of hash tables, the intricacies of hash functions, and the fundamental operations associated with hash tables. Let's recap the key points:

### Structure of Hash Tables

- A hash table is a collection of key-value pairs. It uses a hash function to compute an index into an array of buckets or slots, from which the desired value can be found.
- The efficiency of a hash table depends on the hashing function, the size of the hash table, and the collision resolution method used.
- Collision resolution strategies, such as chaining and open addressing (like linear probing), are essential to handle scenarios where multiple keys hash to the same index.

### Hash Functions

- A hash function takes a key and produces an index in the array, where the corresponding value will reside.
- A good hash function should be fast, distribute keys uniformly across the hash table, and have minimal collisions.
- Complex data types, such as strings, need a specific hash function that first converts the key into an integer.

### Basic Hash Tables Operations

- Hash tables can perform common operations like insertion, deletion, and search.
- Because of hashing, the operations have a time complexity of $O(1)$.
- The actual mechanics of collision resolution take place in the operations themselves.

Hash tables are a critical part of efficient data storage and retrieval. They rely on the robustness of the hash function and the effectiveness of the collision resolution strategy. Understanding these components and their implementation is key to leveraging the full potential of hash tables in applications that require fast and reliable data access. This knowledge forms a foundation for further exploration into more complex data structures and algorithms.

# Formative Assessment 1

# Formative Assessment 2